

Indian Institute of Technology, Patna
Department of Computer Science and Engineering
CS3101: Operating System
Lab Assignment 6 - Concise Study Material

1. Multi-Stage Pipeline with Bounded Buffers

Pipeline Model

A pipeline divides work into stages that execute concurrently:

$$\text{Producers} \rightarrow Q_1 \rightarrow \text{Processors} \rightarrow Q_2 \rightarrow \text{Consumer}.$$

Different items may occupy different stages simultaneously. Every item must retain its `task_id` so it can be verified and reordered later.

Circular Bounded Buffer

A buffer of capacity C maintains an array plus:

`front, rear, count.`

```
/* enqueue */
items[rear] = item;
rear = (rear + 1) % CAPACITY;
count++;

/* dequeue */
item = items[front];
front = (front + 1) % CAPACITY;
count--;
```

All queue-state checks and updates must occur under the queue mutex.

Condition-Variable Pattern

```
pthread_mutex_lock(&q->mutex);
while (q->count == CAPACITY && !shutdown)
    pthread_cond_wait(&q->not_full, &q->mutex);
/* enqueue */
pthread_cond_signal(&q->not_empty);
pthread_mutex_unlock(&q->mutex);
```

`pthread_cond_wait()` atomically releases the mutex while sleeping and reacquires it before returning. Always use `while`: wakeups may be spurious, or another thread may change the queue first.

Termination without Poison Pills

Completion is a state predicate, not a special data value:

- Processor exits when Q_1 is empty and both Producers finished.
- Consumer exits when Q_2 is empty and both Processors finished.
- The last Producer broadcasts to Processors; the last Processor wakes the Consumer.

Update completion counters under a mutex. Test completion inside the same `while` loop used for empty-buffer waiting.

Ordering and Integrity

Parallel Processors may finish out of order. The Consumer can store each result at index `task_id-1`, mark the ID as seen, and write the final log in increasing ID order after collection. Check:

$$\text{produced} = \text{processed} = \text{consumed} = N.$$

Also verify unique IDs, correct squares, empty buffers, thread joins, and exactly N log records.

Use a private `rand_r()` seed per Producer; only the Consumer writes the log. Never hold a queue mutex during calculation, printing, or file I/O.

2. Matrix Multiplication with a POSIX Thread Pool

Why a Thread Pool?

Creating one thread per matrix element causes excessive creation, stack, and scheduling overhead. A fixed pool of T workers is created once and reuses the same threads for transposition and multiplication.

Contiguous Matrix Storage

For a row-major matrix with r rows and c columns:

$$\text{element}(i, j) = \text{array}[i * c + j].$$

Contiguous dynamic allocation is cache-friendly and avoids large stack/VLA objects. Before allocation, check dimension multiplication for `size_t` overflow.

Why Transpose B ?

In the direct formula,

$$C[i][j] = \sum_{p=0}^{K-1} A[i][p]B[p][j],$$

A 's row is contiguous but B 's column is not. With $B^T[j][p] = B[p][j]$, both operands are read sequentially, improving cache locality.

Two Parallel Phases and Barrier

1. Worker t transposes columns $t, t + T, t + 2T, \dots$
2. Every worker calls `pthread_barrier_wait()`.
3. Only after all arrive may multiplication start.

A barrier is a phase boundary, not mutual exclusion. No mutex is needed when workers write disjoint transpose regions or distinct result rows.

Dynamic Row Scheduling

```
pthread_mutex_lock(&scheduler_mutex);
row = next_row++;
pthread_mutex_unlock(&scheduler_mutex);
if (row >= M) break;
/* compute the complete row without the mutex */
```

The critical section assigns work only. Keeping $O(KN)$ row computation outside it permits true parallelism. Each worker records its own row count.

Safe Start Gate

A barrier initialized for T threads deadlocks if only $T - 1$ are created. Therefore workers first wait at a condition-variable start gate:

- if all creations succeed, main sets `start` and broadcasts;
- on failure, main sets `abort`; workers exit before the barrier.

Main then joins created workers and releases all resources.

Verification and Timing

Compute an independent sequential reference from original A, B , compare every result element, and report a checksum. Measure with `CLOCK_MONOTONIC`; parallel timing includes transposition, barrier, and multiplication. A small problem may show speedup below 1 because overhead dominates.

Key idea: Synchronization protects scheduling and phase order; it should not serialize the numerical computation.

Common Mistakes

One thread per cell; wrong row-major stride; omitting the barrier; timing only multiplication in the parallel case; holding the scheduling mutex during row calculation; or failing to handle partial thread creation.

3. Royal Banquet with a Waiter Monitor

Monitor-Based Resource Allocation

A monitor combines shared state, one mutex, and condition variables. In this problem, only the Waiter reserves forks; Philosophers never acquire fork mutexes independently. This removes circular wait and therefore prevents deadlock.

All fork assignments, Philosopher states, tickets, and closure flags are changed under the monitor mutex. Thinking and eating delays occur outside it.

Request and Grant Flow

1. Philosopher locks the monitor, becomes `HUNGRY`, records monotonic request time/ticket, signals the Waiter, and waits on its private condition.
2. Waiter selects an eligible request, atomically reserves two distinct forks, marks it `EATING`, and signals that Philosopher.
3. Philosopher eats without the monitor mutex, then re-locks, releases both forks, updates statistics, and signals the Waiter.

Every condition wait must recheck grant, departure, closure, and error predicates in a `while` loop.

Fair Scheduling

At least two forks are required for a grant.

- A request waiting at least threshold T is *urgent*. Among urgent requests choose earliest request time, then smallest ticket.
- If none is urgent, choose fewest meals, then earliest request, then smallest ticket.

Urgency gives aging-based starvation resistance. It cannot guarantee immediate service while fewer than two forks are free; it guarantees priority at the next feasible grant.

Timed Events

Use `CLOCK_MONOTONIC` for arrival, departure, waiting time, and banquet deadline. A Waiter using only ordinary condition waits may sleep past a deadline if nobody signals; use `pthread_cond_timedwait()` for the next relevant event. Configure the condition clock to monotonic where supported. `nanosleep()` is acceptable only for simulated thinking/eating.

Dynamic Departure and Closure

A hungry Philosopher whose departure time arrives cancels its request and leaves. An eating Philosopher releases both forks before leaving. At banquet deadline, Waiter stops new grants, cancels and broadcasts to all pending requests, then waits for active meals to release resources. Main prints the report only after all joins.

Statistics and Invariants

For each Philosopher record requests, meals, cancellations, threshold crossings, total/average/maximum wait. Continuously check:

- every grant contains two valid, distinct, currently free forks;
- a fork belongs to at most one Philosopher;
- fairness violations remain zero;
- final state has all Philosophers `LEFT` and all forks free.

Key idea: The Waiter decides who may proceed; condition variables block Philosophers until that decision or a timed state change.

Common Mistakes

Philosophers reserving forks themselves; holding the monitor while sleeping; using wall-clock time for elapsed intervals; not waking timed-out requests; confusing threshold crossing with immediate service; or printing the report before active meals return forks.